NEXUS AI RESEARCH LABORATORY | AUDITED & VERIFIED ACROSS 150 EPISODES | MULTI-AGENT RL & SIMULATION

# STRATOS AI: End-to-End Project Engineering Report

Complete Technical Specification, Mathematical Formulation, Architecture & Empirical Results

**Project:** Strategic Team Reasoning and Adaptive Tactical Operations System • **Author Team:** NEXUS AI • **Date:** October 2026

## 1. Executive Summary & Core Research Question

In distributed multi-agent systems, agents making independent decisions often suffer from **coordination collapse**: units fail to synchronize defensive formations, contest resources inefficiently, and leave critical infrastructure exposed. **STRATOS AI** was built to investigate a fundamental scientific question:

> **Core Research Hypothesis:**
> *"Does real-time, structured inter-agent communication measurably improve team coordination, strategic adaptability, and combat win rates compared to independently operating agents across diverse adversarial tactics?"*

To evaluate this rigorously, we developed a 2D real-time strategy simulation adhering to **PettingZoo Parallel API** standards, implemented a heterogeneous squad of three specialized agents (**Commander**, **Defense Guard**, **Attack Striker**), and trained an Actor-Critic policy using **Proximal Policy Optimization (PPO)**. In a multi-seed benchmark of 150 episodes across 5 random seeds, coordinated agents achieved a **100.0% win rate** compared to **66.7%** for independent agents (+33.3% communication advantage), completely eliminating squad wipeouts against aggressive rush attacks (0% → 100%).

## 2. Theoretical Formulation: Dec-POMDP

Our strategic simulation is formally formulated as a **Decentralized Partially Observable Markov Decision Process (Dec-POMDP)** defined by the tuple $\langle \mathcal{N}, \mathcal{S}, \{\mathcal{A}_i\}, \mathcal{P}, \{\mathcal{R}_i\}, \{\Omega_i\}, \mathcal{O}, \gamma \rangle$:

- $\mathcal{N}$ = {commander, defense, attack} is the set of $N = 3$ learning agents.
- $\mathcal{S}$ is the true global game state (all positions, health pools, crystal counts, and timers).
- $\mathcal{A}_i$ is the discrete action space of 7 macro-actions for each agent.
- $\mathcal{P}(s' \mid s, a)$ is the transition dynamics governed by the simulation kinematics and combat rules.
- $\mathcal{R}_i$ is the multi-objective reward signal combining team victory and role alignment.
- $\Omega_i$ is the 49-dimensional continuous observation vector received by agent $i$.
- $\gamma = 0.99$ is the discount factor for cumulative future returns.

## 3. Game Concept & Battlefield Mechanics

The game environment represents a top-down tactical battle on an **800 × 600 pixel coordinate plane**:

- **Friendly Headquarters**: Located at (80, 300) with 1,000 HP. If destroyed, friendly team loses immediately.
- **Enemy Headquarters**: Located at (720, 300) with 1,000 HP. If destroyed, friendly team wins immediately.
- **Crystal Depots**: Three resource nodes at North (400, 150), Center (400, 300), and South (400, 450). Each holds 500 crystals; units within 40px harvest 0.5 crystals/step.
- **Heterogeneous Squad Roles**:

| Agent Role | Max HP | Speed | Damage | Attack Range | Vision Range | Tactical Responsibility |
|---|---|---|---|---|---|---|
| **Commander** | 280 HP | 3.2 px/s | 18.0 HP | 95.0 px | 220.0 px | Strategic macro directives, crystal mining management. |
| **Defense Guard** | 380 HP | 2.6 px/s | 22.0 HP | 85.0 px | 190.0 px | High-armor anchor; shields base, absorbs damage, intercepts threats. |
| **Attack Striker** | 220 HP | 3.8 px/s | 32.0 HP | 120.0 px | 240.0 px | High-damage scout; long firing range, flanks enemies and sieges base. |

### Adversary Tactical Archetypes

1. **Aggressive Rush**: Bypasses crystals; charges directly toward friendly HQ to overwhelm defense early.
2. **Defensive Turtle**: Anchors tightly around enemy HQ (150px perimeter), engaging only when invaded.
3. **Resource-Focused**: Greedily harvests center crystals until stockpiling 100 resources, then launches siege.

## 4. Complete Technical Stack

| Layer | Technology | Role in STRATOS |
|---|---|---|
| **Core Language** | Python 3.11+ | Primary environment, agent logic, and simulation engine. |
| **Deep Learning** | PyTorch ( `torch.nn` , `Adam` ) | Actor-Critic Policy network, PPO loss computation, backpropagation. |
| **MARL Standard** | PettingZoo (Parallel API) & Gymnasium | Multi-agent step execution, decentralized observations, individual rewards. |
| **RL Algorithm** | PPO + GAE-λ | On-policy Actor-Critic with clipped surrogate loss and value estimation. |
| **Visualization** | Pygame | Real-time 60 FPS battlefield canvas, lasers, HUD telemetry, live radio ticker. |
| **Analytics UI** | Streamlit, Pandas, Matplotlib | Interactive research dashboard showing win rates, learning curves, and distributions. |
| **Quality Assurance** | Pytest | 20 automated unit & integration tests passing at 100%. |

## 5. System Architecture & Observation/Action Pipeline

### The 49-Dimensional Continuous Observation Space

Each agent receives a dense vector normalized to `[-1.0, 1.0]`:

- **Channels 0–4 (Self State)**: ($x_{norm}$, $y_{norm}$, HP ratio, cooldown ratio, alive flag)
- **Channels 5–7 (Friendly HQ)**: ($x_{norm}$, $y_{norm}$, HP ratio)
- **Channels 8–10 (Enemy HQ)**: ($x_{norm}$, $y_{norm}$, HP ratio)
- **Channels 11–18 (2 Teammates)**: 2 × (Δx, Δy, HP ratio, alive flag)
- **Channels 19–30 (3 Enemies)**: 3 × (Δx, Δy, HP ratio, alive flag)
- **Channels 31–39 (3 Resource Nodes)**: 3 × (Δx, Δy, crystal ratio)
- **Channels 40–42 (Global State)**: Friendly crystals, enemy crystals, match step ratio
- **Channels 43–48 (Tactical Comm Bus)**: Directive code, threat level, target Δx, target Δy, support flag, message priority

### The 7 Discrete Macro-Actions

`HOLD (0)` • `MOVE_BASE (1)` • `MOVE_ENEMY_BASE (2)` • `MOVE_RESOURCE (3)` • `ATTACK_NEAREST_ENEMY (4)` • `DEFEND_PERIMETER (5)` • `TACTICAL_RETREAT (6)`

## 6. Tactical Communication Bus & 15-Step Freshness Decay

Rather than uninterpretable continuous latent vectors, STRATOS uses a **structured symbolic messaging bus**:

- **Directives Supported**: `ENEMY_DETECTED`, `DEFEND_BASE`, `REQUEST_SUPPORT`, `ATTACK_TARGET`, `RESOURCE_LOW`, `RESUME_OFFENSE`.
- **15-Step Temporal Decay Filter**: In early builds, emergency support requests lingered in mailboxes forever, causing agents to stay in defensive loops even after hostiles died. We implemented a temporal decay filter: messages with `Age > 15 steps` are automatically discarded.
- **Dynamic Command Succession**: If the Commander unit is eliminated, seniority passes to the Defense Guard (then Attack Striker) to maintain squad coordination.

## 7. Reinforcement Learning: PPO Algorithm Details

PPO optimizes a clipped surrogate objective to avoid destructive policy steps:

```
L_CLIP(θ) = E_t [ min( r_t(θ) * A_t,  clip(r_t(θ), 1-ε, 1+ε) * A_t ) ]
```

where $r_t(\theta) = \pi_\theta(a_t|s_t) / \pi_{\theta_{old}}(a_t|s_t)$, $\varepsilon = 0.2$, and $A_t$ is computed via Generalized Advantage Estimation (GAE-λ) with γ = 0.99 and λ = 0.95.

The total loss function minimized by Adam (lr = $3 \times 10^{-4}$) is:

```
L_total = -L_CLIP(θ) + 0.5 * L_VF(ϕ) - 0.01 * Entropy(π_θ)
```

## 8. Empirical Benchmark Results (150 Episodes Across 5 Seeds)

| Scenario / Opponent | Condition | Win Rate | Mean Reward | Survival Rate | Avg Messages |
|---|---|---|---|---|---|
| **Aggressive Rush** | **Independent** | **0 / 25 (0.0%)** | -408.89 | 0.0% (Squad Wipeout) | 0.0 msgs |
| **Aggressive Rush** | **Coordinated** | **25 / 25 (100.0%)** | **+242.26** | **33.3%** | **819.0 msgs** |
| **Defensive Turtle** | Independent | 25 / 25 (100.0%) | +556.86 | 100.0% | 0.0 msgs |
| **Defensive Turtle** | Coordinated | 25 / 25 (100.0%) | +556.86 | 100.0% | 697.0 msgs |
| **Resource Focused** | Independent | 25 / 25 (100.0%) | +408.82 | 33.3% | 0.0 msgs |
| **Resource Focused** | Coordinated | 25 / 25 (100.0%) | +408.82 | 33.3% | 503.0 msgs |

> **Summary Findings Across All 150 Episodes:**
>
> - **Overall Win Rate**: Independent = 66.7% vs **Coordinated = 100.0% (+33.3% advantage)**.
> - **Mean Team Reward**: +185.60 vs **+402.65 (+217.05 points higher)**.
> - **The Aggressive Rush Phenomenon**: Independent agents get wiped out 100% of the time (0% win rate). Coordinated agents broadcast `DEFEND_BASE`, assemble perimeter defenses, and achieve a **100% win rate (+100.0% turnaround)**.

## 9. Real Engineering Obstacles Overcome

1. **The Stale Mailbox Defect**: Emergency support alerts stayed active indefinitely; solved with the 15-step temporal freshness decay filter.
2. **Broken Command Succession**: When Commander died, directives stopped updating; solved with dynamic command delegation to Defense and Attack units.
3. **False Baseline Timeout Diagnosis**: 1,200-step baseline appeared to win against aggressive rushers purely by accidentally wandering away to harvest northern crystals and winning on timeout points. When units were trained to attack, they walked into open fire and died. Fixed with the perimeter guard incentive and widened threat detection.

## 10. How to Run & Verify the Project

- **Interactive Visualizer**: `.\.venv\Scripts\python scripts/run_demo.py` (Press `[M]` to toggle between Coordinated and Independent mode live!)
- **Analytics Dashboard**: `.\.venv\Scripts\streamlit run dashboard/app.py`
- **Automated Test Suite**: `.\.venv\Scripts\pytest -q` (20/20 tests passing in ~11s).

## 11. Tutor Q&A Defense Matrix

**Q1: "Did you just hardcode rules, or is this genuine Reinforcement Learning?"**

"It is genuine Reinforcement Learning. The agents' action selection is governed by an Actor-Critic Neural Network trained via Proximal Policy Optimization (PPO) in PyTorch. The policy receives a 49-dimensional continuous observation vector and outputs probability distributions over 7 actions. What is structured is the communication schema, which feeds directly into observation channels 43 to 48. The policy network learned through PPO updates when to react to those messages."

**Q2: "How do you prove the policy actually uses the communication channels rather than ignoring them?"**

"We conducted a formal Policy Sensitivity Audit on the neural network weights. By keeping physical observations (channels 0–42) completely identical and only modifying the communication channels (43–48), we observed a measurable shift in action logits ($\Delta_{max}$ = 0.0516) and a +0.931 shift in the Critic's state-value estimation.

Furthermore, in our 150-episode benchmark, zeroing out the comm channels caused a 100% squad wipeout against aggressive rushers, while active comm channels delivered a 100% win rate."

**Q3: "Why PettingZoo instead of standard Gym / Gymnasium?"**

"Gymnasium is designed for single-agent systems. PettingZoo is the multi-agent standard, specifically the Parallel API, which allows simultaneous agent execution, decentralized observations, and individualized reward vectors."

**Q4: "Why PPO over DQN or SAC?"**

"DQN struggles with non-stationary multi-agent dynamics because replay buffers contain experiences generated by earlier teammate policies. SAC is designed for continuous action spaces, whereas our macro-actions are discrete. PPO with GAE is on-policy, stable, resilient to hyperparameter variations, and avoids destructive updates via clipped surrogate objectives."